



Profa. Anelise Munaretto

Redes sem Fio — Ciência da Computação

2026

Objetivos de aprendizagem

Ao final desta aula, você será capaz de:

- **descrever** o ciclo agente–ambiente do RL e **diferenciar** aprendizado centralizado de *federated learning*;
- **distinguir** aprendizagem supervisionada, não-supervisionada, por reforço e profunda;
- **aplicar** a equação de Bellman e a atualização Q-learning em alocação de recursos;
- **explicar** o uso de LSTM na predição de mobilidade e *handover*;
- **avaliar** desafios de dados, privacidade, confiabilidade e padronização de ML em redes (3GPP RM-NET, O-RAN).

Pré-requisitos

Aulas 7 (5G e redes móveis) e 10 (drones, satélites e NTN).

Por que IA/ML em redes sem fio?

Crise de complexidade

Redes 5G/6G têm centenas de parâmetros ajustáveis; otimização manual é inviável. IA/ML oferece soluções para:

- predição de tráfego e mobilidade
- alocação dinâmica de recursos (espectro, potência, beam)
- detecção de anomalias e falhas
- gerenciamento autônomo (*zero-touch*)
- otimização cross-layer

Tipos de aprendizado de máquina

Paradigma	Dados	Objetivo	Exemplos em redes sem fio
Supervisionado	Rótulos (entrada $\rightarrow$ saída)	Mapear entrada $\rightarrow$ saída	classificação de tráfego, previsão de RSSI, detecção de interferência
Não-supervisionado	Sem rótulos	Descobrir estrutura oculta	<i>clustering</i> de usuários por mobilidade, compressão de CSI, anomalias
Por reforço	Recompensas (atrasadas)	Maximizar retorno acumulado	<i>handover</i> , alocação de RB, escalonamento, potência

Tipos de aprendizado de máquina

Paradigma	Dados	Objetivo	Exemplos em redes sem fio
Supervisionado	Rótulos (entrada $\rightarrow$ saída)	Mapear entrada $\rightarrow$ saída	classificação de tráfego, previsão de RSSI, detecção de interferência
Não-supervisionado	Sem rótulos	Descobrir estrutura oculta	<i>clustering</i> de usuários por mobilidade, compressão de CSI, anomalias
Por reforço	Recompensas (atrasadas)	Maximizar retorno acumulado	<i>handover</i> , alocação de RB, escalonamento, potência

Ideia-chave

A escolha do paradigma depende do dado disponível: rótulos são caros e escassos em redes (CSI, tráfego); recompensas naturais (throughput, atraso) favorecem o RL.

Aprendizado supervisionado: regressão vs. classificação

Métricas de classificação

Classificação (saída discreta)

- tipo de tráfego (vídeo/voz/dados)
- detecção de intrusão e *jamming*
- reconhecimento de atividade (Wi-Fi sensing)

Regressão (saída contínua)

- predição de throughput e SINR
- estimativa de canal (CSI) e de posição
- tempo até o *handover*

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precisão} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}.$$

Métrica de regressão

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Atenção

Anomalias em redes são raras $\Rightarrow$ a acurácia engana; priorize *recall* e F_1 .

Overfitting e validação de modelos

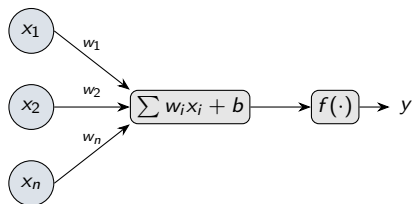
Overfitting

O modelo memoriza o treino e falha ao generalizar: excelente no treino, ruim em dados novos — típico com redes profundas e poucos exemplos rotulados.

Conjunto	Uso	Armadilha
Treino	ajustar pesos	usar validação no treino vaza informação
Validação	escolher hiperparâmetros	reusar a mesma validação muitas vezes
Teste	avaliar a generalização (uma vez!)	nunca treinar com o teste

- Em redes, a distribuição dos dados muda no tempo/espço (mobilidade, hora do dia) $\Rightarrow$ use **validação temporal** e re-treino periódico.
- Ferramentas: *k*-fold cross-validation, *early stopping*, regularização L1/L2 e *dropout*.

Neurônio artificial e funções de ativação



$$z = \sum_i w_i x_i + b, \quad y = f(z).$$

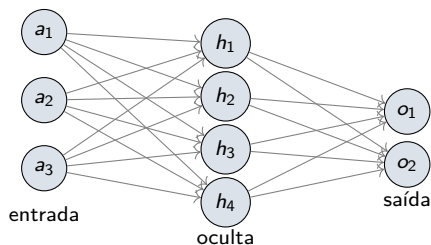
Funções de ativação

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$ — clássica para classificação binária; satura e desvanece o gradiente.
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$ — a mais usada em redes profundas; não satura para $z > 0$.
- **Perceptron:** neurônio com ativação degrau (saída binária) — base histórica.

No contexto de RF

Entradas típicas: medições de potência (RSRP), CQI, espectrogramas; o neurônio computa uma combinação linear e a ativação introduz a não-linearidade.

MLP e *backpropagation*



- **MLP:** camadas de neurônios com não-linearidades empilhadas $\Rightarrow$ *aproximador universal* de funções.
- **Backpropagation:** o erro da saída é propagado para trás (regra da cadeia) e calcula o gradiente de cada peso; o SGD atualiza $w \leftarrow w - \eta \nabla_w \mathcal{L}$.
- As camadas **ocultas** extraem *features* intermediárias automaticamente.

Em redes sem fio

Classificação de CSI, mapeamento sinal $\rightarrow$ localização, estimativa de CQI a partir de séries de RSSI.

Por que *deep learning* funciona bem para RF?

- **Padrões ricos:** o canal sem fio tem correlações espaço-temporais (multipath, *fading*, interferência) que redes profundas capturam hierarquicamente.
- **Convoluções** (CNN): capturam correlações locais em espectrogramas e grades de CSI.
- **Recorrência** (LSTM/GRU): modelam a evolução temporal do canal e do tráfego.
- **Autoencoders:** comprimem o CSI para *feedback* eficiente (reduz overhead de sinalização).
- **Aprendizado de *features*:** dispensa a engenharia manual de características, que exigia profundo conhecimento de RF.

Mas...

DL exige volume de dados, é caro computacionalmente e difícil de interpretar — por isso modelos híbridos e *physics-informed* estão em alta.

Aprendizagem Supervisionada

- classificação de interferência
- predição de SNR/CQI
- detecção de tipo de tráfego

Aprendizagem Não-Supervisionada

- agrupamento de padrões de mobilidade
- compressão de CSI

Aprendizagem por Reforço (RL)

- *resource block allocation*
- escalonamento de pacotes
- controle de admissão

Aprendizagem Profunda (DL)

- *channel estimation* com autoencoders
- beamforming com CNN
- predição de séries temporais (LSTM)

RL para alocação de recursos



Figura 1: ciclo agente–ambiente do aprendizado por reforço aplicado à alocação de recursos.

- Exemplo: alocação de RBs em 5G usando Deep Q-Learning (DQN)
- Estado: filas CQI, buffer; Ação: RBs por usuário; Recompensa: throughput $\times$ justiça (Jain)

Exemplo prático: predição de mobilidade

- **Problema:** antecipar handover para evitar perda de pacotes
- **Método:** LSTM treinado com sequências de RSSI/RSRP
- **Entrada:** série temporal de 10 medições de potência (5 vizinhos)
- **Saída:** célula-alvo e tempo estimado de handover
- **Resultado:** redução de 40% em falhas de handover (trabalhos reais)

Implementação

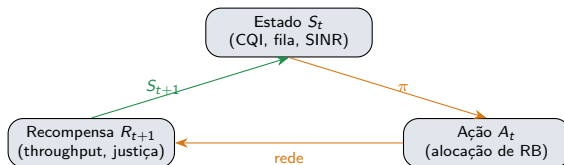
Python + TensorFlow/Keras, treino offline com dados de drive-test, inferência no MEC ou no próprio UE.

MDP formal: o modelo matemático do RL

Um MDP é a 5-tupla (S, A, P, R, γ) :

- S : **estados** (CQI, filas, SINR);
- A : **ações** (RBs, célula-alvo, potência);
- $P(s'|s, a)$: probabilidade de **transição**;
- $R(s, a)$: **recompensa** imediata;
- $\gamma \in [0, 1)$: fator de **desconto**.

Uma **política** $\pi(a|s)$ mapeia estados em ações; objetivo: retorno $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1}$.

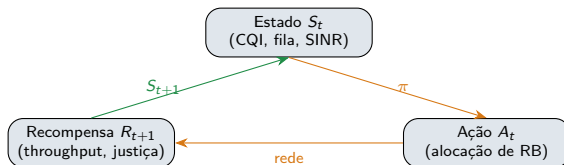


MDP formal: o modelo matemático do RL

Um MDP é a 5-tupla (S, A, P, R, γ) :

- S : **estados** (CQI, filas, SINR);
- A : **ações** (RBs, célula-alvo, potência);
- $P(s'|s, a)$: probabilidade de **transição**;
- $R(s, a)$: **recompensa** imediata;
- $\gamma \in [0, 1)$: fator de **desconto**.

Uma **política** $\pi(a|s)$ mapeia estados em ações; objetivo: retorno $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1}$.



Equação de Bellman (forma Q)

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

onde r é a recompensa imediata; $\max_{a'} Q(s', a')$ é o melhor valor futuro a partir de s'

Exploração vs. exploração e a política ε -greedy

- **Exploitation:** escolher a ação com maior Q estimado — bom agora, mas pode ser um ótimo local.
- **Exploration:** tentar ações ainda mal avaliadas — descobre recompensas melhores no futuro.

Política ε -greedy

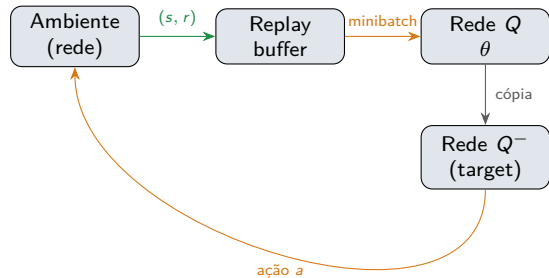
Com probabilidade ε escolhe ação **aleatória** (explora); com $1 - \varepsilon$ escolhe a melhor ação:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|} & \text{se } a = \arg \max_{a'} Q(s, a'), \\ \frac{\varepsilon}{|A|} & \text{caso contrário.} \end{cases}$$

- **Decay de ε :** começa alto (ex.: $\varepsilon = 1,0$) e decai (ex.: $\times 0,99$ por episódio) $\Rightarrow$ explora muito no início e converge para o *greedy* no fim.
- Em redes reais, explorar demais degrada a QoS do usuário; sistemas vivos usam ε pequeno ou exploram no simulador.

DQN: experiência replay e target network

- **Problema da tabela Q :** estados contínuos (CSI, filas) tornam a tabela inviável.
- **DQN:** a rede neural $Q(s, a; \theta)$ aproxima o valor de cada ação.
- **Experiência replay:** guarda (s, a, r, s') num buffer e amostra minibatches $\Rightarrow$ quebra correlação temporal.
- **Target network:** rede Q^- congelada, atualizada periodicamente $\Rightarrow$ o alvo não oscila a cada passo, **estabilizando o treino.**



Alvo do DQN

$$y = r + \gamma \max_{a'} Q^-(s', a')$$

Aprendizado por reforço profundo multiagente (MARL)

Cenário

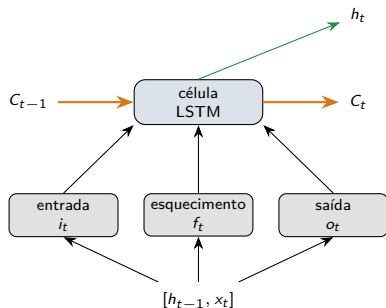
Vários agentes (células, gNBs, usuários) otimizam juntos: cada um aprende sua política, mas o ambiente depende das ações de *todos*.

- **Não-estacionariedade:** quando outro agente muda a política, as transições observadas mudam $\Rightarrow$ o problema de otimização se desloca.
- **Desafios:** convergência (os alvos se movem), coordenação entre agentes, escalabilidade e custo de comunicação.

Técnicas e exemplo

- *Centralized training / decentralized execution* (CTDE): treina com informação global, executa só com observação local.
- Exemplo: **handover coordenado** entre células vizinhas — cada gNB decide a transferência considerando a carga das vizinhas.

LSTM: memória de longo prazo



$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh C_t.$$

- O estado de célula C_t carrega a **memória de longo prazo**; os portões controlam o fluxo de informação.
- Portão de **esquecimento** (f_t): o que descartar do passado.
- Portão de **entrada** (i_t): o que gravar do novo dado.
- Portão de **saída** (o_t): o que expor como estado oculto h_t .

Em redes

Ideal para sequências de RSSI/RSRP, contagem de pacotes e CSI ao longo do tempo.

Previsão de tráfego e mobilidade: ARIMA, LSTM e Transformers

Modelo	Tipo	Vantagens	Limitações
ARIMA	estatístico linear	simples, interpretável, rápido	assume estacionariedade; ignora não-linearidades
LSTM	DL recorrente	não-linear; memória longa	treino sequencial; exige mais dados
Transformer	DL com atenção	paralelismo; dependências longas	caro; exige muitos dados (popular pós-2020)

Aplicações

- **Dimensionamento:** prever picos de tráfego por célula e alocar capacidade.
- **QoS preditivo:** antecipar degradação de SINR/atraso e reagir proativamente.
- **Mobilidade:** prever célula-alvo e instante do *handover* (ex.: LSTM sobre RSRP) e reduzir falhas de handover.

Detecção de anomalias: jamming, intrusão e falhas

Anomalias típicas em RF

- **Jamming**: elevação súbita da potência / erro de decodificação;
- **Intrusão**: padrões de tráfego atípicos (varredura, botnet);
- **Falhas**: queda de desempenho, rádio com mau funcionamento.

Técnicas

- **Autoencoder**: reconstrói o dado “normal”; erro alto $\Rightarrow$ anomalia.
- **One-class SVM**: aprende a fronteira dos dados normais; ponto fora $\Rightarrow$ anomalia.
- **LSTM preditivo**: erro de predição grande sinaliza evento anômalo.

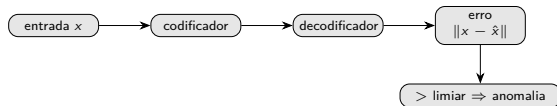
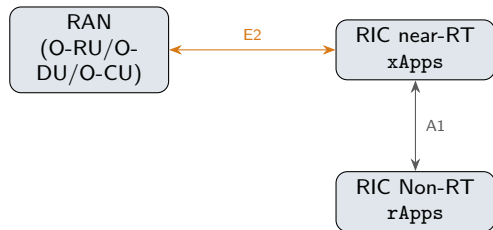


Figura 2: autoencoder treinado só com dados normais; desvios são detectados pela reconstrução.

O-RAN: RAN aberta e inteligente

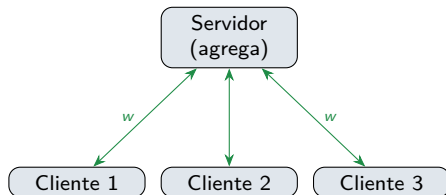


- **RIC near-real-time:** loop de 10 ms a 1 s; hospeda **xApps** para controle fino (beam, *handover*, escalonamento).
- **RIC non-real-time:** loop > 1 s; hospeda **rApps** para otimização, treino de modelos e políticas.
- **Interfaces:** A1 (política), E2 (controle/telemetria da RAN) e O1 (gestão); o O-RAN define um AI/ML workflow.

Casos de ML

Beam management, seleção de *handover*, alocação de recursos e detecção de falhas — implementados como xApp/rApp com dados da RAN.

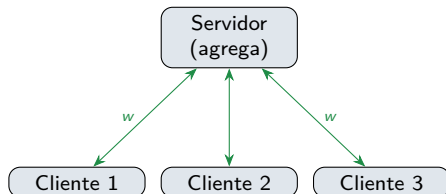
Federated learning: FedAvg



Algoritmo FedAvg (passos)

- 1 o servidor envia o modelo inicial w ;
- 2 cada cliente treina **localmente** com seus dados (sem enviá-los);
- 3 os clientes devolvem os **pesos** (ou gradientes);
- 4 o servidor agrega pela média ponderada:
$$w \leftarrow \sum_k \frac{n_k}{n} w_k;$$
- 5 repete por várias rodadas.

Federated learning: FedAvg



Algoritmo FedAvg (passos)

- 1 o servidor envia o modelo inicial w ;
- 2 cada cliente treina **localmente** com seus dados (sem enviá-los);
- 3 os clientes devolvem os **pesos** (ou gradientes);
- 4 o servidor agrega pela média ponderada:
$$w \leftarrow \sum_k \frac{n_k}{n} w_k;$$
- 5 repete por várias rodadas.

Privacidade e custo

Os dados nunca saem do cliente (privacidade/GDPR); porém há custo de comunicação (modelos grandes), clientes heterogêneos e dados não-IID.

Split learning, edge AI e IA generativa

Split learning e inferência na borda

- **Split learning**: a rede é dividida — camadas iniciais no dispositivo, finais no servidor; só *ativações intermediárias* trafegam (menos sensível que dados brutos).
- **Edge AI**: inferência no nó de borda (MEC/gNB/UE) $\Rightarrow$ baixa latência; modelos compactados com **quantização** (FP32 $\rightarrow$ INT8), *pruning* e *distillation*.
- Trade-off: **latência vs. acurácia** vs. privacidade.

IA generativa em redes (2024–2026)

- **GANs**: geram dados sintéticos de tráfego/canal $\Rightarrow$ aumentam datasets rotulados escassos e preservam privacidade.
- **LLMs**: assistentes de operação — diagnóstico de falhas, geração de configuração e tradução de políticas; tema ativo de pesquisa (IA *native* para 6G).

Beamforming com ML

- ML prevê o canal (CSI) ou diretamente o melhor feixe do *codebook*.
- CNN sobre CSI estima o feixe ótimo sem varrer todos $\Rightarrow$ reduz o **overhead de sondagem** em mmWave/6G.
- Aplicações: 5G *beam management*, ondas milimétricas.

Posicionamento com ML

- *Fingerprinting*: mapa de assinaturas de canal (CSI/RSRP) por localização.
- Regressão/classificação (CNN, LSTM) sobre CSI estima a posição $\Rightarrow$ precisão sub-métrica em *indoor*.
- Vantagem sobre o GPS: funciona *indoor* e em ambientes urbanos densos.

Fio condutor

Nos dois casos, o ML extrai informação de **características físicas do rádio** (CSI) que seriam difíceis de modelar analiticamente.

Scheduling com RL

- **Estado:** CQI por usuário, tamanho das filas;
- **Ação:** alocação de *resource blocks* (RBs) aos usuários;
- **Recompensa:** throughput e justiça combinados, ex.: índice de Jain

$$J = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2};$$

- **Objetivo:** equilibrar eficiência e justiça (proporcionalidade).

Handover com DRL

- **Estado:** SINR atual e das vizinhas, carga das células, velocidade;
- **Ação:** célula-alvo (ou adiar/adiantar o *handover*);
- **Recompensa:** QoE — throughput, atraso e número de *ping-pong*;
- DQN aprende a política melhor que disparos fixos (evento A3); re-treino quando o ambiente muda.

Dados

- coleta de dados rotulados (CSI, tráfego)
- privacidade (GDPR, nuvem vs. borda)
- distribuição não-estacionária (mobilidade)

Complexidade

- inferência em tempo real (latência)
- tamanho do modelo (memória do dispositivo)
- *federated learning* para escalar

Confiabilidade

- modelos precisam generalizar
- interpretabilidade (explainable AI)
- robustez a ataques adversários

Padronização

- 3GPP Rel. 18: RM-NET (ML para RAN)
- O-RAN Alliance: rApp e xApp com ML
- ITU-T ML5G

Desafios de ML em redes: além dos dados

Dados e confiança

- rótulos escassos e caros (anotar tráfego/CSI);
- **interpretabilidade (XAI)**: a operadora precisa justificar decisões (SHAP, LIME);
- robustez a dados não-estacionários (mobilidade, carga).

Custo computacional

- latência de inferência em tempo real na borda;
- energia do treinamento de modelos profundos (*green AI*).

Segurança adversarial

- perturbações imperceptíveis enganam classificadores (evasão em IDS);
- envenenamento de dados em *federated learning*;
- *model inversion* sobre modelos treinados.

Padrões em formação

- 3GPP Rel. 18/19 RM-NET e *AI/ML for NG-RAN*;
- O-RAN AI/ML workflow e ITU-T ML5G (Y.3172);
- interoperabilidade e conformidade ainda imaturas.

Exercício resolvido — Bellman e atualização Q-learning

Problema: um agente RL decide a alocação de um RB. A ação atual não gera recompensa imediata ($r = 0$), mas leva a um estado em que a melhor ação futura renderá $r = 10$ (estado terminal). Considere $\gamma = 0,9$ e partida com $Q(s, a) = 0$.

❶ **Valor-alvo (Bellman):**

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') = 0 + 0,9 \times 10 = 9.$$

❷ **Atualização Q-learning** com $\alpha = 0,1$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] = 0 + 0,1 (9 - 0) = 0,9.$$

❸ **Convergência:** repetindo a mesma experiência, $Q(s, a)$ se aproxima assintoticamente de 9.

Interpretação

O fator $\gamma = 0,9$ desconta o valor futuro: uma recompensa de 10 a um passo de distância vale $0,9 \times 10 = 9$ hoje.

- **ML em redes:** predição de tráfego/mobilidade, alocação dinâmica de espectro/potência/beam, detecção de anomalias, *zero-touch*.
- **Paradigmas:** supervisionado (classificação), não-supervisionado (agrupamento), por reforço (alocação) e profundo (LSTM, CNN, autoencoders).
- **RL/DQN:** agente aloca RBs com base em estado (CQI, buffer) e recompensa (throughput $\times$ justiça de Jain); DQN usa redes neurais como aproximador de Q .
- **LSTM:** predição de *handover* a partir de séries de RSSI/RSRP (redução de $\approx 40\%$ de falhas).
- **Desafios:** dados rotulados, privacidade (GDPR), não-estacionariedade, latência de inferência e robustez.
- **Padronização:** 3GPP Rel. 18 RM-NET, O-RAN (rApp/xApp) e ITU-T ML5G; *federated learning* para escalar com privacidade.

Autoavaliação

- 1 No aprendizado por reforço, o agente obtém do ambiente:
- (a) apenas o rótulo correto da ação
 - (b) apenas a ação executada
 - (c) o novo estado e a recompensa

- ① No aprendizado por reforço, o agente obtém do ambiente:
- (a) apenas o rótulo correto da ação
 - (b) apenas a ação executada
 - (c) o novo estado e a recompensa

Gabarito: (c)

- ② Uma LSTM é mais adequada para:
- (a) predição de séries temporais, como RSSI/RSRP
 - (b) ordenação de pacotes em filas
 - (c) compressão de imagens de beamforming

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

Autoavaliação

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

Gabarito: (b)

❹ Com $\gamma = 0,9$, $r = 0$ e $\max Q(s', a') = 10$, a equação de Bellman resulta em $Q(s, a) =$

- (a) 9

(b) 10

Autoavaliação

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

Gabarito: (b)

❹ Com $\gamma = 0,9$, $r = 0$ e $\max Q(s', a') = 10$, a equação de Bellman resulta em $Q(s, a) =$

- (a) 9

(b) 10

- ① Classificar o tipo de tráfego (vídeo/voz/dados) a partir de estatísticas de fluxo é um exemplo de:
- (a) aprendizado supervisionado (classificação)
 - (b) aprendizado não-supervisionado
 - (c) aprendizado por reforço

- ① Classificar o tipo de tráfego (vídeo/voz/dados) a partir de estatísticas de fluxo é um exemplo de:
- (a) aprendizado supervisionado (classificação)
 - (b) aprendizado não-supervisionado
 - (c) aprendizado por reforço

Gabarito: (a)

- ② No DQN, a função da *target network* é:
- (a) reduzir o tamanho da rede neural
 - (b) estabilizar o treinamento com um alvo que não muda a cada passo
 - (c) gerar dados sintéticos de treinamento

- ① Classificar o tipo de tráfego (vídeo/voz/dados) a partir de estatísticas de fluxo é um exemplo de:
- (a) aprendizado supervisionado (classificação)
 - (b) aprendizado não-supervisionado
 - (c) aprendizado por reforço

Gabarito: (a)

- ② No DQN, a função da *target network* é:
- (a) reduzir o tamanho da rede neural
 - (b) estabilizar o treinamento com um alvo que não muda a cada passo
 - (c) gerar dados sintéticos de treinamento

Gabarito: (b)

Exercício: tabela Q com ε -greedy

Dois estados S_1, S_2 , duas ações A_1, A_2 ; $\gamma = 0,9$, $\alpha = 0,1$, $\varepsilon = 0,5$ (decai), tabela zerada.

❶ **Passo 1:** em S_1 escolhe A_2 (exploração) $\rightarrow$ recompensa $r = 1$, novo estado S_2 :

$$Q(S_1, A_2) \leftarrow 0 + 0,1[1 + 0,9 \cdot \max(0, 0) - 0] = 0,1.$$

❷ **Passo 2:** em S_2 escolhe A_1 (exploração) $\rightarrow r = 0$, novo estado S_1 :

$$Q(S_2, A_1) \leftarrow 0 + 0,1[0 + 0,9 \cdot \max(0; 0,1) - 0] = 0,009.$$

❸ **Passo 3:** em S_1 escolhe A_1 (*greedy*) $\rightarrow r = 5$, novo estado S_2 :

$$Q(S_1, A_1) \leftarrow 0 + 0,1[5 + 0,9 \cdot \max(0,009; 0) - 0] \approx 0,50.$$

Exercício: tabela Q com ε -greedy

Dois estados S_1, S_2 , duas ações A_1, A_2 ; $\gamma = 0,9$, $\alpha = 0,1$, $\varepsilon = 0,5$ (decai), tabela zerada.

❶ **Passo 1:** em S_1 escolhe A_2 (exploração) $\rightarrow$ recompensa $r = 1$, novo estado S_2 :

$$Q(S_1, A_2) \leftarrow 0 + 0,1[1 + 0,9 \cdot \max(0, 0) - 0] = 0,1.$$

❷ **Passo 2:** em S_2 escolhe A_1 (exploração) $\rightarrow r = 0$, novo estado S_1 :

$$Q(S_2, A_1) \leftarrow 0 + 0,1[0 + 0,9 \cdot \max(0; 0,1) - 0] = 0,009.$$

❸ **Passo 3:** em S_1 escolhe A_1 (*greedy*) $\rightarrow r = 5$, novo estado S_2 :

$$Q(S_1, A_1) \leftarrow 0 + 0,1[5 + 0,9 \cdot \max(0,009; 0) - 0] \approx 0,50.$$

Tabela Q resultante

	A_1	A_2
S_1	0,50	0,10
S_2	0,009	0,00

Q-learning em Python/gymnasium

- Implementar Q-learning para alocação de canal/RB em um ambiente simplificado (ex.: FrozenLake ou um env customizado).
- Variar α e γ e observar a curva de recompensa acumulada e a convergência da tabela Q .
- Extensão: substituir a tabela por uma rede neural (DQN) usando PyTorch/TensorFlow.

Demonstração de *federated learning*

- Usar tensorflow-federated (TFF) para treinar um classificador com clientes simulados.
- Comparar precisão e custo de comunicação entre treino centralizado e federado.

- C. Zhang et al., “Deep learning in mobile and wireless networking: A survey”, *IEEE Commun. Surv. Tut.*, 2019
- N. C. Luong et al., “Applications of deep reinforcement learning in communications and networking”, *IEEE Commun. Mag.*, 2019
- 3GPP TR 38.817, *Study on RAN-centric data collection and utilization (RAN-CU)*
- O-RAN Alliance, *O-RAN Working Group 2: AI/ML Workflow Description*
- ITU-T Y.3172, *Architectural framework for ML in future networks*

Leitura recomendada

- R. Sutton & A. Barto, *Reinforcement Learning: An Introduction*, MIT Press — referência clássica para RL, Bellman e Q-learning.
- C. Zhang et al., “Deep learning in mobile and wireless networking: A survey”, *IEEE Commun. Surv. Tut.*, 2019 — panorama geral de DL aplicado a redes.
- N. C. Luong et al., “Applications of deep reinforcement learning in communications and networking”, *IEEE Commun. Mag.*, 2019 — DQN aplicado a alocação de recursos.
- O-RAN Alliance, *WG2: AI/ML Workflow Description* — como o ML entra na RAN (rApp/xApp).
- 3GPP TR 38.817 (RAN-CU) — coleta e uso de dados centrados na RAN para ML.

Dica de estudo

Reimplemente o exercício de Bellman em Python e depois generalize para DQN; isso consolida o ciclo agente–ambiente.